Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3698038.3698546

RESEARCH-ARTICLE

The Hidden Carbon Footprint of Serverless Computing

ROHAN BASU ROY

RAGHAVENDRA KANAKAGIRI, Indian Institute of Technology Tirupati, Tirupati, AP, India

YANKAI JIANG

DEVESH TIWARI

Open Access Support provided by:

Indian Institute of Technology Tirupati

PDF Download
3698038.3698546.pdf
02 March 2026
Total Citations: 7
Total Downloads: 1091

Published: 20 November 2024

Citation in BibTeX format

SoCC '24: ACM Symposium on Cloud Computing
November 20 - 22, 2024
WA, Redmond, USA

Conference Sponsors:
SIGOPS
SIGMOD

SoCC '24: Proceedings of the 2024 ACM Symposium on Cloud Computing (November 2024)
https://doi.org/10.1145/3698038.3698546
ISBN: 9798400712869

The Hidden Carbon Footprint of Serverless Computing

Rohan Basu Roy
Northeastern University

Raghavendra Kanakagiri
Indian Institute of Technology Tirupati

Yankai Jiang
Northeastern University

Devesh Tiwari
Northeastern University

ABSTRACT

Due to the unique aspects of serverless computing like keep-alive and co-location of functions, it is challenging to account for its carbon footprint. This is the first work to introduce the need for systematic methodologies for carbon accounting in the serverless environment, propose new methodologies and in-depth analysis, and highlight how the carbon footprint estimation can vary based on the chosen methodology. It discusses how serverless-specific scheduling choices can impact the trade-offs between performance and carbon footprint, with an aim toward standardizing methodological choices and identifying opportunities for future improvements.

CCS CONCEPTS

• **Computer systems organization** → Cloud computing.

KEYWORDS

Serverless Computing, Carbon Footprint, Sustainability

ACM Reference Format:
Rohan Basu Roy, Raghavendra Kanakagiri, Yankai Jiang, and Devesh Tiwari. 2024. The Hidden Carbon Footprint of Serverless Computing. In *ACM Symposium on Cloud Computing (SoCC '24), November 20–22, 2024, Redmond, WA, USA.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3698038.3698546

1 Introduction And Background

Carbon emissions of computing systems. The greenhouse gas emissions from large-scale computing systems constitute more than 2.5% of global emissions [23, 29]. With the increasing demand for computing resources, the carbon footprint of computing is also increasing rapidly. Industries and cloud computing service providers are increasingly aiming to enhance the sustainability of their operations, focusing on reducing carbon emissions during the manufacturing of computing systems, energy consumption during operations, and using renewable energy during operations.

The carbon footprint of computing systems comprises the *operational* and *embodied* carbon. The *operational carbon* is the carbon emitted during the hardware operation, for the energy required for operation. It is a product of the energy consumption and the *carbon intensity* (CI). The CI is the amount of carbon emitted per unit of generated energy from a specific source (*i.e.,* if a power grid uses more renewable sources at a time, it delivers energy with lower CI). The CI varies temporally and spatially depending on the energy mix being used by a power grid of a location. The *embodied carbon* is the carbon emitted during the manufacturing of hardware, and it is amortized over the lifetime of hardware. Due to the effect of embodied carbon during manufacturing and varying levels of carbon intensity, minimizing carbon footprint is not the same as minimizing energy consumption, as also acknowledged by prior studies [21, 23] – although minimizing energy consumption is certainly helpful in also reducing carbon footprint.

Serverless computing and its carbon footprint. Serverless computing is a rapidly increasing form of cloud computing, with more than 70% of users of production cloud platforms like AWS, using one or more serverless solutions [11]. Since serverless functions are typically short-running, providers *keep alive* functions in the memory of servers when they are predicted to be invoked. If a function is not kept alive and it is invoked, it undergoes a *cold start*, otherwise, it undergoes a *warm start*. A higher keep-alive period increases the warm start chances. The summation of the execution time and the cold start time (0, if warm start) is the *service time* of a function.

While there have been attempts to measure and model the carbon footprint of applications running on a dedicated environment, they cannot be directly translated to estimate the carbon footprint of serverless computing. In this work, we argue that serverless computing has considerable environmental implications. While the stateless nature of serverless computing is attractive, the need to keep alive functions in memory incurs high embodied carbon emissions and should be accurately and robustly accounted for. Furthermore, the

This work is licensed under a Creative Commons Attribution International 4.0 License.
SoCC '24, November 20–22, 2024, Redmond, WA, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1286-9/24/11.
https://doi.org/10.1145/3698038.3698546

virtue of statelessness allows cloud providers to execute functions on different heterogeneous servers. However, the choice of server for keeping the function alive, the keep-alive duration, and execution location significantly impact the carbon footprint – often at odds with serverless performance with respect to function service time.

We demonstrate that the carbon intensity plays a significant role in the overall carbon footprint of serverless functions – especially, as we move toward renewable energy sources, hence, the impact of the keep-alive period on the overall carbon footprint will only increase – highlighting the increasing impact of this problem. This paper also makes some thought-provoking observations which hopefully, will spark more community-wide discussion and engagement. For example, older generation hardware has lower embodied carbon and hence, results in a low keep-alive carbon footprint. Due to this, we argue that service providers may prefer to keep alive functions on older generation hardware in some cases, but place these hardware in low carbon intensity regions to leverage the benefit of warm start and reduce the operational carbon due to the low carbon intensity – this argument is also demonstrated in the first carbon footprint-focused serverless computing work [25].

This work also establishes that the magnitude and relative contribution of embodied and operational carbon footprint in serverless depends on the chosen methodology. We do not claim that the methodological choices presented in this paper are the only reasonable choices, but instead, we hope that they initiate a discussion on the standardization of methodological choices for carbon accounting and measurements in serverless computing platforms.

Contributions. We make the following contributions:

1. The fundamental intent of this paper is to attract our community's attention toward a critical problem – the environmental impact of the serverless computing model – a problem that is likely to grow further.

2. We demonstrate that "unique" aspects of serverless computing (e.g., *proactively keeping a function alive in memory to increase the chances of warm start for lower latency*) make carbon footprint accounting and measurement more challenging from the methodological point of view.

3. This study also revealed several new insights – the influence of methodological choices on overall carbon footprint estimation of serverless computing, the high impact of embodied carbon during the keep-alive period on overall carbon footprint (due to the short-running nature of serverless functions and the high embodied carbon of memory), the trade-off between performance and carbon footprint, and impact of function executions on different hardware choices.

2 Function Keep-Alive And Execution Carbon Footprint Accounting And Measurement

In this section, we discuss methodologies to measure and account for carbon emissions during function execution and keep-alive periods, highlight associated challenges, and identify areas needing more attention.

2.1 Carbon Footprint Accounting

Here, we discuss the methodology for accounting for carbon footprint during keep-alive and function execution, considering both embodied and operational carbon in each phase.

For serverless functions, accounting for the carbon footprint is particularly challenging and methodologically error-prone because of the unique keep-alive period aspects of serverless functions. There are multiple important details to be considered when accounting for the overall carbon footprint – especially, the embodied carbon footprint during the keep-alive period.

2.1.1 Keep-Alive Carbon Footprint Accounting. During the keep-alive period, functions are kept alive in the memory of the servers for the entire duration [40]. This results in DRAM energy consumption and utilization. Thus, there is both operational and embodied carbon consumption of the DRAM, which should be accounted as:

$$C_{f_{\text{keep-alive}}} = \underbrace{\frac{E_{\text{DRAM}}}{L_{\text{DRAM}}} * T_{\text{keep-alive}}}_{\text{Embodied}} + \underbrace{\text{CI} * J_{\text{DRAM}}}_{\text{Operational}} \quad (1)$$

Here, the carbon footprint of a serverless function f during the keep-alive period ($C_{f_{\text{keep-alive}}}$) depends on the total embodied carbon of the DRAM (E_{DRAM}), the lifetime of the DRAM (L_{DRAM}), the keep-alive time ($T_{\text{keep-alive}}$), the carbon intensity (CI) and the energy consumption of the DRAM (J_{DRAM}) during the keep-alive period. While this accounting model is simple, it pessimistically assumes that only one function is kept alive in the memory of the server, with no other functions executing or being kept alive in the server. However, that is often not the case due to multi-tenancy. Hence, the embodied carbon of the DRAM must be shared between all the active functions (kept alive or currently executing) in proportion to the amount of memory they consume.

$$C_{f_{\text{keep-alive}}} = \underbrace{\frac{E_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}{L_{\text{DRAM}}} * T_{\text{keep-alive}}}_{\text{Embodied}} + \underbrace{\text{CI} * J_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}_{\text{Operational}} \quad (2)$$

Here, $\Delta_{f_{\text{DRAM}}}$ is the average of the fraction of memory consumed by function f compared to all other active functions, during the entire keep-alive period. In this accounting model, we implicitly assume that the energy consumption, resulting

in the operational carbon footprint, is divided in proportion to the amount of memory consumed by the function f, compared to other active functions. Note that each active function's actual DRAM energy consumption depends on the amount of memory and the number of memory read/writes and refresh operations.

A key challenge in serverless keep-alive carbon accounting is that DRAM energy cannot be measured per co-located functions, as current hardware aggregates power at the memory package level. Fine-grained monitoring at the level of individual functions would require instrumentation capable of distinguishing between different memory access sources.

> *Given that memory and storage are known to have significant embodied carbon, failure to account for the embodied carbon footprint of DRAM during the keep-alive period can lead to a significant underestimation of the environmental implication of serverless computing.*

> *One may be tempted to assume that functions are typically kept-alive on harvested memory capacity which may have gone unutilized anyway and hence, it should not account for serverless functions. We caution that this is problematic since the use of these harvested resources leads to lower serverless service time (due to the avoidance of cold start). Hence, their leveraged benefit must be accounted toward carbon footprint estimation – just like keep-alive memory consumption is accounted toward keep-alive cost in serverless research [5, 37].*

Next, an important consideration is accounting for the embodied and operational carbon of CPU cores. A naive approach would be to exclude the CPU core footprint, assuming they aren't used during keep-alive. However, this is methodologically flawed, as some CPU cores must remain reserved in anticipation of function invocation. *The purpose of keep-alive is defeated if a CPU core is unavailable at invocation.*

Another counter-argument is to be "reactive" and make CPU cores available after invocation – this indeed naively suggests removing the liability to account for the carbon footprint of CPUs during keep-alive. Unfortunately, the "reactive" approach requires preemption of existing workloads running CPU cores which incurs latency overhead, again defeating the purpose of keeping alive a function in memory. The "reactive" approach also limits the choices of workloads that can be run (i.e., they need to be eligible for preemption at all times) – indirectly equivalent to "reserving" the cores for only a limited type of activity and provisioning for how many such cores to choose pessimistically.

In summary, we argue and envision a complete specification of CPU core carbon accounting during the keep-alive period. One potential methodology is to account for the embodied carbon of the CPU as follows.

$$C_{f_{\text{keep-alive}}} = \Big(\underbrace{\frac{E_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}{L_{\text{DRAM}}}}_{\text{Embodied DRAM}} + \overbrace{\frac{E_{\text{CPU}} * \frac{n_f}{N_{\text{CPU}}}}{L_{\text{CPU}}}}^{\text{Embodied CPU}} \Big) * T_{\text{keep-alive}} + \underbrace{\text{CI} * J_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}_{\text{Operational}} \tag{3}$$

Here, E_{CPU} is the embodied carbon of the entire CPU package, L_{CPU} is the lifetime of the CPU package, N_{CPU} is the total number of cores, and n_f is the number of cores reserved for function f (this is for reservation purposes only and does not need to be the same as the number of cores the function may need during execution). In general, n_f is proportional to the amount of allocated memory for a serverless function. For example, in the AWS Lambda serverless platform, users can select up to 10GB of memory to be allocated to their functions, and proportionately the number of allocated vCPUS scales between one and six [39]. Note that, in this formulation, the embodied carbon of the uncore components scales in proportion to the number of allocated cores.

The cores that cannot be used during the keep-alive period of functions, still consume idle power, even when they are not actively executing functions. This idle power results in an operational carbon footprint. Hence, the carbon footprint of a function f during the keep-alive period is the following:

$$C_{f_{\text{keep-alive}}} = \Big(\overbrace{\frac{E_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}{L_{\text{DRAM}}}}^{\text{Embodied DRAM}} + \overbrace{\frac{E_{\text{CPU}} * \frac{n_f}{N_{\text{CPU}}}}{L_{\text{CPU}}}}^{\text{Embodied CPU}} \Big) * T_{\text{keep-alive}} + \text{CI} * \Big(\underbrace{J_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}_{\text{Operational DRAM}} + \underbrace{J_{\text{CPU}} * \lambda_{\text{IDLE}} * \frac{n_f}{N_{\text{CPU}}}}_{\text{Operational CPU}} \Big) \tag{4}$$

Here, J_{CPU} is the energy consumption of the entire CPU package during the keep-alive period. CPU energy cannot be measured per core because the Model-Specific Registers (MSRs) used for power measurement, such as Intel's RAPL (Running Average Power Limit), typically provide energy readings for the entire CPU package rather than individual cores. Note that the idle power consumption of the cores that cannot be used for function execution and remain reserved to keep-alive function f is not the same as the power consumption of the cores executing functions. This is why J_{CPU} cannot simply be multiplied by $\frac{n_f}{n_{\text{CPU}}}$ to determine the energy consumption of the idle cores. We define the idle power factor (λ_{IDLE}) as the ratio of the average utilization of the idle cores reserved for the function f and the average utilization of all the cores in the CPU package, during the keep-alive period of f. Multiplying λ_{IDLE} with J_{CPU} and $\frac{n_f}{n_{\text{CPU}}}$ results in the power consumption of the idle cores.

The carbon footprint for serverless functions during the keep-alive period must account for both the DRAM and CPU cores – although one may naively assume that CPU cores are not actively used during the keep-alive period and hence, they do not account for the embodied or operational carbon footprint. The CPU cores are indirectly consumed – unfortunately, their accounting is more challenging than DRAM resources because the "implicit reservation" of CPU cores can be different across service providers and may not always have a linear correspondence with the explicitly reserved DRAM capacity.

2.1.2 Function Execution Carbon Footprint Accounting. Next, we discuss the carbon footprint of serverless functions during their execution. Accounting for carbon footprint during function execution is relatively simpler compared to the keep-alive period carbon footprint accounting because this phase (function execution) shares similarities with the execution of regular workloads and hence, a similar methodology can be adopted. During the function execution, service providers allocate a certain number of cores to a function, based on the amount of memory requested by the user [39]. We note that serverless functions are stateless by nature and they can be scheduled on different types of CPU and DRAM hardware. Therefore, the accounting must specify the exact hardware type, including the CPU and DRAM type.

The stateless nature of serverless functions allows scheduling flexibility but makes the function execution carbon footprint accounting more challenging – requiring exact specification of the underlying hardware, especially when comparing different techniques and when the execution platforms may be a mix of heterogeneous hardware.

The service providers also allocate a fixed amount of ephemeral storage (512 MB in AWS Lambda) per function during execution. The embodied and operational carbon footprint of DRAM, CPU, and storage constitutes the carbon footprint during execution.

$$C_{f_{\text{execution}}} = \Big(\underbrace{\frac{E_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}{L_{\text{DRAM}}}}_{\text{Embodied DRAM}} + \underbrace{\frac{E_{\text{CPU}} * \frac{n_f}{N_{\text{CPU}}}}{L_{\text{CPU}}}}_{\text{Embodied CPU}} + \underbrace{\frac{E_{\text{ST}} * \frac{S_f}{S_{\text{ST}}}}{L_{\text{ST}}}}_{\text{Embodied Storage}} \Big) * T_{\text{exe}} + \text{CI} * \Big(\underbrace{J_{\text{DRAM}} * \Delta_{f_{\text{DRAM}}}}_{\text{Operational DRAM}} + \underbrace{J_{\text{CPU}} * \frac{n_f}{N_{\text{CPU}}}}_{\text{Operational CPU}} + \underbrace{J_{\text{ST}} * \frac{S_f}{S_{\text{ST}}}}_{\text{Operational Storage}} \Big) \tag{5}$$

Here, E_{ST} is the embodied carbon of the storage drive, and L_{ST} is its lifetime. $\frac{S_f}{S_{\text{ST}}}$ is the fraction of the storage drive space allocated for function f compared to the entire size of the drive. The execution time of the function, which includes the cold start time (in case of a cold start), is denoted by T_{exe}. The energy consumption of the storage during the function execution is denoted by J_{ST}.

This definition of carbon footprint assumes that the energy consumption of all the functions co-located in the entire system is the same. However, in reality, since a node can co-locate functions that are executing, and functions that are just being kept alive (reserving respective cores), the energy consumption of different cores can be different. Thus, similar to the idle power factor (introduced in Sec. 2.1.1), we define the execution factor (λ_{exe}), which is the ratio of the average utilization of the cores used for executing function f and the average utilization of all the cores in the CPU package, during the execution period of f. This gives a more accurate representation of energy consumption and the operational carbon footprint of the CPU. Thus, $J_{\text{CPU}} * \lambda_{\text{exe}} * \frac{n_f}{N_{\text{CPU}}}$ is a more accurate representation of the energy consumption by the CPU package during to the execution of function f.

Similar to the varying energy consumption of the CPU for different co-located functions, the DRAM and storage energy consumption also vary, but such variations and their effect on the overall carbon footprint are small. Also, accounting for those variations will require more function-level profiling to determine their memory and I/O operations, which can further increase the service time of functions. This is why, we do not account for such variations in the formulation.

The short-running nature of serverless functions and their relatively low CPU and memory requirements make them ideal for co-location. Unfortunately, co-location makes accounting for carbon footprint more challenging since the operational carbon estimation requires precise energy consumption – highlighting the need for future research in this area.

The summation of the execution and keep-alive carbon footprint is the total carbon footprint of a serverless function. We note that carbon intensity plays a major role in determining the total carbon footprint, as it impacts both execution and keep-alive carbon footprints. Carbon intensity varies spatially and temporally [34]. Note that here we account for carbon footprint at the function level as it enables cloud providers to make precise scheduling decisions by selecting optimal hardware and data center locations to minimize embodied and operational carbon based on each function's resource needs. This function-level accounting can extend to the resource level by aggregating carbon footprint across all functions and distributing it proportionally based on memory, CPU, and storage consumption. Next, we discuss the methodology to measure the actual carbon footprint by using hardware counters and open-sourced information.

2.2 Carbon Footprint Measurement

2.2.1 Measuring (estimating) the embodied carbon footprint. Vendors like Dell and Apple have open-sourced embodied carbon data for their hardware – including DRAM, CPU package, storage, and accelerators [3, 44]. Estimating embodied carbon typically involves detailed life cycle assessments (LCAs) that account for emissions from the entire lifecycle of a product, including material extraction, manufacturing, transportation, use, and end-of-life disposal. Aggregating data from these sources, there are open-sourced datasets that report embodied carbon of different server-grade bare-metal platforms and VMs of major cloud providers [4, 7, 12]. Typically, it is assumed that the lifetime of hardware is four or five years [19, 41]. The embodied carbon of storage drives needs to be accounted for from vendor product sheets [12, 30].

2.2.2 Measuring operation carbon footprint. Measuring the operational carbon footprint requires measuring the energy consumption. Intel provides the RAPL interface with MSR registers to measure the energy consumption of DRAM and a CPU package (core and uncore components) at the socket level. Also, using high-level APIs like Likwid [46], one can measure energy consumption by accessing the register values. Since energy consumption cannot be measured at a per-function level, the relative average fraction of memory consumption ($\Delta_{f_{\text{DRAM}}}$), number of allocated cores (n_f), the idle power factor (λ_{IDLE}), and execution factor (λ_{exe}) (discussed in Sec. 2.1.1 and Sec. 2.1.2) can be used to estimate the carbon emission at a per-function level from the energy data of the entire DRAM and CPU package. Note that the package-level energy consumption excludes the energy consumption of the storage drives. Typically, product specifications of storage drives report the power consumption for different levels of I/O operations [38]. Since the energy consumption of the storage drive is significantly lower than that of the cores and DRAM, assuming a specific level of I/O operations for the entire duration of function execution, results in very little variations (less than 0.3%) in the accounting of carbon footprint. CI data of different power grids is made available at an hourly granularity through APIs like Electricity Maps [34].

2.2.3 Measuring carbon footprint in shared cloud Environment. Serverless researchers and developers set up serverless execution frameworks to execute serverless functions as containers or microVMs in a cluster of bare-metal nodes [1, 13, 16, 37, 49]. Even in production cloud platforms like AWS, MSR registers can be accessed in bare-metal machines, thus making it possible to measure energy consumption and carbon footprint. However, production serverless platforms, like AWS Lambda, Google Cloud Functions, and Azure Functions, do not expose these registers to users. When an experiment is required to be performed in these production serverless platforms, researchers need to perform one-time profiling of their functions on bare-metal instances to measure energy consumption. Next, we utilize these carbon accounting methodologies, to discuss some opportunities to reduce serverless carbon emissions.

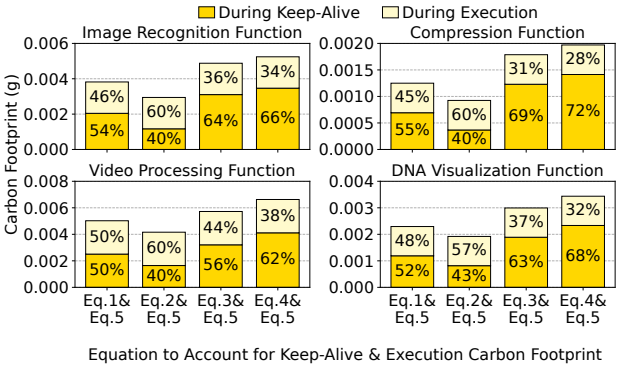

Figure 1: Keep-alive carbon footprint forms a major part of the total carbon footprint. Depending upon the chosen methodology, the total carbon footprint estimation and the relative contribution from keep-alive and execution carbon footprints can vary to a great extent (CI = 345, execution and keep-alive on AWS c5 server, with a ten-minute keep-alive).

3 Implications Of Methodological Choices And Experimental Setup

Here, we discuss the implications of different methodological choices and scheduling on the serverless carbon footprint.

3.1 Impact of Keep-Alive Carbon Footprint And Methodological Choices

In Sec. 2.1.1, through Eq. 1-4, we discuss the different methodological choices to account for the carbon footprint during the keep-alive period. Accounting for carbon during the execution of a function is similar to carbon footprint estimation of other workloads, and is formulated in Eq. 5. In Fig. 1, we observe the measured carbon footprint during the execution and a ten-minute keep-alive period (used frequently in production environments [40]) for Image Recognition, Compression, Video Processing, and DNA Visualization functions from SeBS benchmark suite [10], which is widely used in serverless research [5, 37, 43, 49]. The functions are kept alive and executed on a c5 bare-metal EC2 instance from the AWS cloud in the region `us-east-2`. The carbon intensity is set to be 345 gCO_2/kWh (average in 2023 [34]).

First, we note that the keep-alive carbon contributes to a major portion (more than 50%, in most cases) of the total carbon footprint of serverless functions. This is because serverless functions are short-running (a few seconds, in most cases). However, they are kept alive for longer durations

(in the order of tens of minutes) [37, 40]. Keeping functions alive is a serverless-specific phenomenon, requiring future research to reduce its carbon impact. Additionally, we note that due to the methodological differences in accounting for the keep-alive carbon footprint, the total carbon footprint and the relative measures of carbon footprint during execution and keep-alive vary. The absolute value of the carbon footprint during execution remains the same (Eq. 5).

Both Eq. 1 and Eq. 2 account for the embodied and operational carbon of the DRAM, while accounting for the keep-alive carbon footprint. However, Eq. 2 considers the multi-tenancy of functions, and hence the share of embodied and operational carbon of DRAM for individual functions decreases, resulting in a decrease in the keep-alive carbon footprint. In Eq. 3, we additionally account for the embodied carbon of the CPU to measure the keep-alive carbon footprint, due to which the keep-alive carbon footprint increases more than both the previous two accounting methodologies. Finally, in Eq. 4, we also consider the operational carbon of the CPU, in proportion to the CPU utilization by the functions that are kept alive. Thus, the keep-alive carbon increases further beyond all the other methodologies. Since Eq. 4 offers the most comprehensive keep-alive carbon footprint measurement methodology, we use it for keep-alive carbon measurement and Eq. 5 for function execution carbon footprint measurement in the rest of the analyses presented in this paper. Due to the different accounting methodologies, the total carbon footprint measurements vary to a large extent. Note that evaluating the robustness of a carbon footprint model is challenging when vendors do not publish key data on embodied carbon of different hardware components. Variations in hardware lifespan, regional carbon intensity, and the absence of detailed function-level energy data further complicate accurate carbon accounting.

A significant portion of the carbon footprint stems from the keep-alive aspect. In most cases, the carbon footprint during function execution can be even less than the carbon footprint during keep-alive due to inherent short execution times of functions (in the order of a few seconds), but relatively longer keep-alive periods (in the order of tens of minutes).

The overall carbon footprint estimation is sensitive to accounting methodology. In particular, ignoring the embodied carbon of DRAM during keep-alive can result in a significant underestimation of the overall carbon footprint. Accounting for embodied and operational carbon of CPU cores during keep-alive is challenging, and appropriately accounting for these factors further increases the overall carbon footprint of serverless functions.

Figure 2: The carbon footprint varies across different hardware (CI = 345, with a ten-minute keep-alive).

3.2 Impact Of Hardware Choices

Since serverless functions are stateless and executed on hardware determined by the cloud provider, the serverless community has developed several solutions to schedule them across heterogeneous hardware to optimize for metrics like service time and keep-alive cost [5, 6, 36, 37]. However, function execution and keep-alive in heterogeneous hardware can result in very different carbon footprints due to the difference in embodied carbon and energy efficiency of different hardware. Fig. 2 demonstrates this point. Also, the optimal hardware to minimize carbon footprint varies for different serverless functions (*e.g.*, c5 is the carbon-optimal choice for Image Recognition, while m6i is the carbon-optimal choice for Compression function). Additionally, energy-optimal hardware (which minimizes operational carbon) is different from carbon-optimal hardware, due to the impact of embodied carbon footprint [21, 23]. For example, for the Compression function, c6a is the energy-optimal choice, while m6i is the carbon-optimal choice. In this case, serverless functions are kept alive for ten minutes after their execution, and the carbon footprint during the keep-alive period is measured based on Eq. 4. The different heterogeneous hardware are AWS bare-metal EC2 instances in the `us-east-2` region. The embodied carbon footprint of different instances varies due to differences in the manufacturing processes, material sourcing, and energy mix used during the production of the hardware itself. This, combined with the operational carbon from varying energy consumption of a function across different hardware, results in varying overall carbon footprints across different hardware options.

The hardware type significantly affects the carbon footprints of serverless functions. The carbon-optimal hardware type can be different for different functions. Also, as expected, carbon-optimal and energy-optimal choices are often different. Scheduling in heterogeneous hardware can bring in an opportunity to reduce the carbon footprint of serverless computing.

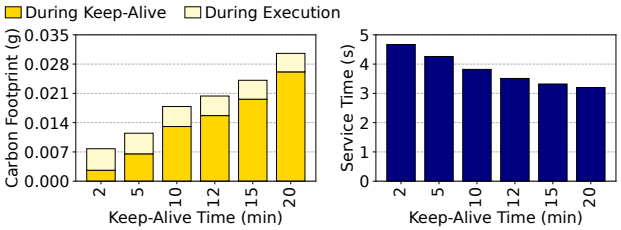

Figure 3: Carbon footprint increases and service time decreases with an increase in keep-alive time (Image Recognition, CI = 345, on AWS m5 server).

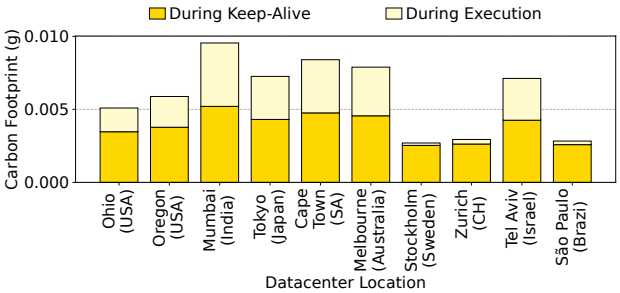

Figure 4: Carbon footprint of execution and keep-alive varies across different datacenter locations, due to variations in carbon intensity (Image Recognition, with a ten-minute keep-alive period, on AWS c5 server).

3.3 Impact Of Keep-Alive Time Choices

From Sec. 2.1.1, we observe that the carbon footprint during the keep-alive period is directly proportional to the keep-alive time. However, a higher keep-alive time increases the chance of a function having a warm start, thus decreasing the service time. From Fig. 3, we observe the divergent trends of mean service time and carbon footprint for a serverless function invocation and keep-alive, based on setting different keep-alive times of a function after each invocation. With an increase in keep-alive time, the carbon footprint increases due to an increase in keep-alive carbon, and service time decreases due to more invocations having warm starts. In this case, the function invocations are performed in an AWS m5 instance, following the Azure serverless function invocation trace [40], for a total duration of two weeks. The keep-alive carbon footprint is calculated based on Eq. 4. This trade-off between carbon footprint and service time, upon varying keep-alive time, demonstrates that there is an opportunity to jointly minimize the carbon emissions, along with traditional performance metrics like the service time.

The keep-alive period significantly affects the keep-alive carbon footprint. A larger keep-alive period is necessary for achieving lower service time (due to more chances of a warm start), but increases carbon footprint – in other words, performance and carbon footprint are at odds.

3.4 Impact Of Carbon Intensity

From Sec. 2.1.1 and Sec. 2.1.2, we note that both keep-alive and execution carbon footprint is directly proportional to the carbon intensity, due to the effect of operational carbon. The carbon intensity varies spatially across different locations due to the different energy sources used in different power grids [34]. In Fig. 4, we execute the serverless function Image Recognition with a ten-minute keep-alive period on an AWS c5 server, across different AWS regions. We use the average carbon intensity values of the respective regions in 2023 to estimate the carbon footprint. We note that due to the difference in CI, the carbon footprint can vary by more than 70% between the location with the lowest carbon footprint (Stockholm) and the one with the highest carbon footprint (Mumbai). Stockholm power grid, which mostly uses renewable energy sources has an average carbon intensity of 35. In contrast, Mumbai, where coal is mainly used for energy generation, has an average carbon intensity of 920 in 2023 [34]. In locations with renewable sources (lower carbon intensity), the carbon footprint during the keep-alive period dominates to a greater extent as the majority of the keep-alive carbon footprint comes from the embodied carbon, while the majority of execution carbon footprint is due to the operational carbon (which is low in locations with renewable sources). For example, in Stockholm, the keep-alive carbon is 94%, while in Mumbai, it is 54% of the total carbon footprint.

Power grid energy mixes change over time, causing carbon intensity to vary temporally [34]. For example, Oregon's carbon intensity fluctuated by over 75% in 2023. Serverless functions, being stateless, can be scheduled across datacenters based on carbon intensity, datacenter load, and transmission latency to optimize for carbon and performance metrics.

The carbon intensity plays a significant role toward the overall carbon footprint of serverless functions – especially as we move toward renewable energy sources (i.e., lower operational carbon), the impact of the keep-alive period on overall carbon footprint will increase because keep-alive period results in high embodied carbon (due to high memory usages for a long time).

3.5 Recommendations for Cloud Service Providers and Policymakers

Location-wise carbon intensity variations and differences in embodied carbon and energy efficiency of hardware can lead to interesting choices for hardware placements in datacenters across different locations to reduce carbon footprint. Embodied carbon of hardware forms the majority of keep-alive carbon, while operational carbon of hardware forms the majority of execution carbon (Sec. 2). In general, older generation hardware has lower embodied carbon, and newer generation hardware has higher embodied carbon and

greater energy efficiency (hence, lesser operational carbon, given a carbon intensity) [4, 12, 21, 48]. An implication of this trend is that service providers may prefer to keep alive functions on older generation hardware but place the older generation hardware in low carbon intensity datacenter locations to leverage the benefit of warm start and reduce the operational carbon due to the low carbon intensity of the region. On the other hand, it is preferable to place newer hardware (energy efficient) in datacenter locations with high carbon intensity so that the operational carbon footprint is not too high due to the energy efficiency of the hardware.

From a function scheduler design point of view, functions that require a higher amount of resources for keeping alive should be placed in older generation hardware so that they incur less embodied carbon during the keep-alive period. Functions with higher energy consumption should be scheduled in newer hardware as they are more energy efficient (the function execution will incur a lesser operational carbon footprint). Additionally, the maximum allowed keep-alive time of functions kept alive in newer hardware should be lower than functions kept alive in older hardware, as the embodied carbon incurred during the keep-alive period is lower in older hardware. Thus, the scheduling of functions for execution across different generations of hardware and the keep-alive time should be adjusted based on the resource requirements, and energy consumption, along with traditional performance metrics like service time of functions.

Policymakers must establish a standardized methodology for carbon accounting in serverless, particularly in multi-tenant environments where embodied carbon must be proportionately shared. Mandatory reporting of both operational and embodied carbon, for DRAM, storage, and CPU, is essential for all providers. Increased transparency in reporting hardware-specific embodied carbon will improve accuracy and help to minimize the environmental impact.

Service providers should use older generation hardware, with lower embodied carbon, for keeping functions alive, especially in regions with low carbon intensity to minimize both embodied and operational carbon footprints. In contrast, newer, energy-efficient hardware should be used in high carbon intensity regions for function execution to reduce operational carbon. Functions with high keep-alive resource needs should be placed on older hardware, while those with high energy consumption should use newer hardware. Shorter keep-alive durations are better for newer hardware to limit embodied carbon impact. Additionally, policymakers should standardize carbon accounting and transparent reporting to improve carbon footprint optimizations in serverless environments.

4 Related Works

Computing sustainability research has focused on reducing energy consumption and the carbon footprint of datacenters via solutions like resource allocation, vertical scaling, workload migration, scheduling, and distributions based on varying carbon intensity [2, 8, 17, 22, 24, 27, 28, 31, 33, 47]. Prior research works have explored power modeling and monitoring techniques to estimate the energy consumption and carbon footprint of cores and virtualized environments [9, 14, 18, 26]. CarbonScaler [24] dynamically varies server allocation based on fluctuations in the carbon intensity of the grid. Carbon Containers [45] optimize for energy efficiency and performance within a predefined carbon budget via a combination of vertical scaling, migration and suspend/resume.

These solutions mainly address operational carbon emissions using simple power models suited for traditional datacenters. However, the dynamic nature of serverless platforms and the impact of keep-alive periods present unique challenges. In a serverless context, EcoFaas [42] uses dynamic voltage and frequency scaling for energy management, while EcoLife [25] co-optimizes carbon and performance with multi-generation hardware, and Caribou [20] reduces serverless workflow's carbon emissions through geospatial shifting. Researchers have explored efficient server design using carbon-efficient compute units [15, 35, 47]. While there have been works on modeling embodied and operational carbon footprint [21], they can only be directly used for workloads running in dedicated environments [30, 32]. Accounting for carbon in serverless is challenging due to serverless-specific phenomena like keep-alive, and co-located executions.

5 Conclusion

This paper formulates several methodological choices for carbon accounting in serverless, each resulting in significantly different carbon footprint estimations. It shows how different scheduling choices of serverless functions can be utilized to reduce carbon footprint. The vision of this paper is to motivate future research on standardizing carbon accounting methodologies in serverless computing, particularly during the keep-alive period, where variations in hardware types and carbon intensity significantly affect overall emissions. This will involve improving estimation accuracy, incorporating both embodied and operational carbon, and designing scheduling strategies to reduce the carbon footprint. Additionally, future research should consider optimizing for both performance and sustainability, ensuring cross-platform consistency, and enabling service providers to make informed, carbon-efficient decisions.

Acknowledgement. We thank Francisco Romero (our shepherd), and the reviewers for their constructive feedback. This work is supported by NSF Awards 1910601 and 2124897.

REFERENCES

[1] Alexandru Agache, Marc Brooker, Alexandra Iordache, Anthony Liguori, Rolf Neugebauer, Phil Piwonka, and Diana-Maria Popa. 2020. Firecracker: Lightweight virtualization for serverless applications. In *17th {usenix} symposium on networked systems design and implementation ({nsdi} 20)*. 419–434.

[2] Thomas Anderson, Adam Belay, Mosharaf Chowdhury, Asaf Cidon, and Irene Zhang. 2023. Treehouse: A case for carbon-aware datacenter software. *ACM SIGENERGY Energy Informatics Review* 3, 3 (2023), 64–70.

[3] Apple. 2024. Apple. Product Environmental Reports. https://www.apple.com/environment/.

[4] Clément Auger, Benoit Hilloulin, Benjamin Boisserie, Maël Thomas, Quentin Guignard, and Emmanuel Rozière. 2021. Open-source carbon footprint estimator: Development and university declination. *Sustainability* 13, 8 (2021), 4315.

[5] Rohan Basu Roy, Tirthak Patel, Rohan Garg, and Devesh Tiwari. 2024. CodeCrunch: Improving Serverless Performance via Function Compression and Cost-Aware Warmup Location Optimization. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 85–101.

[6] Vivek M Bhasi, Aakash Sharma, Shruti Mohanty, Mahmut Taylan Kandemir, and Chita R Das. 2024. Paldia: Enabling SLO-Compliant and Cost-Effective Serverless Computing on Heterogeneous Hardware. In *2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE Computer Society, 100–113.

[7] Sarah B Boyd. 2011. *Life-cycle assessment of semiconductors*. Springer Science & Business Media.

[8] Mohak Chadha, Thandayuthapani Subramanian, Eishi Arima, Michael Gerndt, Martin Schulz, and Osama Abboud. 2023. GreenCourier: Carbon-Aware Scheduling for Serverless Functions. In *Proceedings of the 9th International Workshop on Serverless Computing* (<conf-loc>, <city>Bologna</city>, <country>Italy</country>, </conf-loc>) *(WoSC '23)*. Association for Computing Machinery, New York, NY, USA, 18–23. https://doi.org/10.1145/3631295.3631396

[9] Maxime Colmant, Mascha Kurpicz, Pascal Felber, Loïc Huertas, Romain Rouvoy, and Anita Sobe. 2015. Process-level power estimation in VM-based systems. In *Proceedings of the Tenth European Conference on Computer Systems* (Bordeaux, France) *(EuroSys '15)*. Association for Computing Machinery, New York, NY, USA, Article 14, 14 pages. https://doi.org/10.1145/2741948.2741971

[10] Marcin Copik, Grzegorz Kwasniewski, Maciej Besta, Michal Podstawski, and Torsten Hoefler. 2021. Sebs: A serverless benchmark suite for function-as-a-service computing. In *Proceedings of the 22nd International Middleware Conference*. 64–78.

[11] Datadog. 2023. The State of Serverless — datadoghq.com. https://www.datadoghq.com/state-of-serverless/.

[12] Benjamin Davy. 2021. Building an AWS EC2 carbon emissions dataset. *Medium, September* (2021).

[13] Dong Du, Tianyi Yu, Yubin Xia, Binyu Zang, Guanglu Yan, Chenggang Qin, Qixuan Wu, and Haibo Chen. 2020. Catalyzer: Sub-millisecond startup for serverless computing with initialization-less booting. In *Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems*. 467–481.

[14] G. Fieni, R. Rouvoy, and L. Seinturier. 2020. SmartWatts: Self-Calibrating Software-Defined Power Meter for Containers. In *2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*. IEEE Computer Society, Los Alamitos, CA, USA, 479–488. https://doi.org/10.1109/CCGrid49817.2020.00-45

[15] Eitan Frachtenberg, Ali Heydari, Harry Li, Amir Michael, Jacob Na, Avery Nisbet, and Pierluigi Sarti. 2011. High-efficiency server design. In *Proceedings of 2011 International Conference for High Performance Computing, Networking, Storage and Analysis* (Seattle, Washington) *(SC '11)*. Association for Computing Machinery, New York, NY, USA, Article 27, 27 pages. https://doi.org/10.1145/2063384.2063420

[16] Alexander Fuerst and Prateek Sharma. 2021. FaasCache: keeping serverless computing alive with greedy-dual caching. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 386–400.

[17] Wedan Emmanuel Gnibga, Andrew A Chien, Anne Blavette, and Anne Cécile Orgerie. 2024. FlexCoolDC: Datacenter Cooling Flexibility for Harmonizing Water, Energy, Carbon, and Cost Trade-offs. In *Proceedings of the 15th ACM International Conference on Future and Sustainable Energy Systems*. 108–122.

[18] Bhavishya Goel, Sally A. McKee, Roberto Gioiosa, Karan Singh, Major Bhadauria, and Marco Cesati. 2010. Portable, scalable, per-core power estimation for intelligent resource management. In *International Conference on Green Computing*. 135–146. https://doi.org/10.1109/GREENCOMP.2010.5598313

[19] Sriram Govindan, Anand Sivasubramaniam, and Bhuvan Urgaonkar. 2011. Benefits and limitations of tapping into stored energy for datacenters. In *Proceedings of the 38th annual international symposium on Computer architecture*. 341–352.

[20] Viktor Gsteiger, Pin Hong Daniel Long, Yiran Jerry Sun, Parshan Javanrood, and Mohammad Shahrad. 2024. Caribou: Fine-Grained Geospatial Shifting of Serverless Applications for Sustainability. (2024).

[21] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2022. ACT: Designing sustainable computer systems with an architectural carbon modeling tool. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 784–799.

[22] Udit Gupta, Mariam Elgamal, Gage Hills, Gu-Yeon Wei, Hsien-Hsin S Lee, David Brooks, and Carole-Jean Wu. 2023. Architectural CO 2 Footprint Tool: Designing Sustainable Computer Systems With an Architectural Carbon Modeling Tool. *IEEE Micro* 43, 4 (2023), 107–117.

[23] Udit Gupta, Young Geun Kim, Sylvia Lee, Jordan Tse, Hsien-Hsin S Lee, Gu-Yeon Wei, David Brooks, and Carole-Jean Wu. 2021. Chasing carbon: The elusive environmental footprint of computing. In *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 854–867.

[24] Walid A Hanafy, Qianlin Liang, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. CarbonScaler: Leveraging Cloud Workload Elasticity for Optimizing Carbon-Efficiency. *arXiv preprint arXiv:2302.08681* (2023).

[25] Yankai Jiang, Rohan Basu Roy, Baolin Li, and Devesh Tiwari. 2024. EcoLife: Carbon-Aware Serverless Function Scheduling for Sustainable Computing. *arXiv preprint arXiv:2409.02085* (2024).

[26] Alok Kamatar, Valerie Hayot-Sasson, Yadu Babuji, Andre Bauer, Gourav Rattihalli, Ninad Hogade, Dejan Milojicic, Kyle Chard, and Ian Foster. 2024. GreenFaaS: Maximizing Energy Efficiency of HPC Workloads with FaaS. arXiv:2406.17710 [cs.DC] https://arxiv.org/abs/2406.17710

[27] Young Geun Kim, Udit Gupta, Andrew McCrabb, Yonglak Son, Valeria Bertacco, David Brooks, and Carole-Jean Wu. 2023. GreenScale: Carbon-Aware Systems for Edge Computing. *arXiv preprint arXiv:2304.00404* (2023).

[28] Sven Köhler, Benedict Herzog, Henriette Hofmeier, Manuel Vögele, Lukas Wenzel, Andreas Polze, and Timo Hönig. 2023. Carbon-Aware Memory Placement. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems*. 1–7.

[29] Hessam Lavi. 2022. Measuring greenhouse gas emissions in data centres: the environmental impact of cloud computing | Insights & Sustainability | Climatiq — climatiq.io. https://www.climatiq.io/blog/measure-greenhouse-gas-emissions-carbon-data-centres-cloud-computing.

[30] Baolin Li, Rohan Basu Roy, Daniel Wang, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Toward Sustainable HPC: Carbon Footprint Estimation and Environmental Implications of HPC Systems. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[31] Baolin Li, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Clover: Toward Sustainable AI with Carbon-Aware Machine Learning Inference Service. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.

[32] Baolin Li, Siddharth Samsi, Vijay Gadepally, and Devesh Tiwari. 2023. Sustainable hpc: Modeling, characterization, and implications of carbon footprint in modern hpc systems. *arXiv preprint arXiv:2306.13177* (2023).

[33] Zhenhua Liu, Minghong Lin, Adam Wierman, Steven H Low, and Lachlan LH Andrew. 2011. Greening geographical load balancing. *ACM SIGMETRICS Performance Evaluation Review* 39, 1 (2011), 193–204.

[34] Electricity Maps. 2023. *Reduce carbon emissions with actionable electricity data*. https://www.electricitymaps.com/

[35] Amirhossein Mirhosseini, Akshitha Sriraman, and Thomas F. Wenisch. 2019. Enhancing Server Efficiency in the Face of Killer Microseconds. In *2019 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 185–198. https://doi.org/10.1109/HPCA.2019.00037

[36] Francisco Romero, Mark Zhao, Neeraja J Yadwadkar, and Christos Kozyrakis. 2021. Llama: A heterogeneous & serverless framework for auto-tuning video analytics pipelines. In *Proceedings of the ACM Symposium on Cloud Computing*. 1–17.

[37] Rohan Basu Roy, Tirthak Patel, and Devesh Tiwari. 2022. IceBreaker: warming serverless functions better with heterogeneity. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 753–767.

[38] V Samsung and NAND SSD. 2018. 860 QVO. (2018).

[39] Peter Sbarski and Sam Kroonenburg. 2017. *Serverless architectures on AWS: with examples using Aws Lambda*. Simon and Schuster.

[40] Mohammad Shahrad, Rodrigo Fonseca, Íñigo Goiri, Gohar Chaudhry, Paul Batum, Jason Cooke, Eduardo Laureano, Colby Tresness, Mark Russinovich, and Ricardo Bianchini. 2020. Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. In *2020 {USENIX} Annual Technical Conference ({USENIX}{ATC} 20)*. 205–218.

[41] Arman Shehabi, Sarah Smith, Dale Sartor, Richard Brown, Magnus Herrlin, Jonathan Koomey, Eric Masanet, Nathaniel Horner, Inês Azevedo, and William Lintner. 2016. United states data center energy usage report. (2016).

[42] Jovan Stojkovic, Nikoleta Iliakopoulou, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2024. EcoFaaS: Rethinking the Design of Serverless Environments for Energy Efficiency. In *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA'24)*.

[43] Jovan Stojkovic, Tianyin Xu, Hubertus Franke, and Josep Torrellas. 2023. Mxfaas: Resource sharing in serverless environments for parallelism and efficiency. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. 1–15.

[44] Markus Stutz, Scott O'Connell, and John Pflueger. 2012. Carbon footprint of a dell rack server. In *2012 Electronics Goes Green 2012+*. IEEE, 1–5.

[45] John Thiede, Noman Bashir, David Irwin, and Prashant Shenoy. 2023. Carbon Containers: A System-level Facility for Managing Application-level Carbon Emissions. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) *(SoCC '23)*. Association for Computing Machinery, New York, NY, USA, 17–31. https://doi.org/10.1145/3620678.3624644

[46] Jan Treibig, Georg Hager, and Gerhard Wellein. 2010. Likwid: A lightweight performance-oriented tool suite for x86 multicore environments. In *2010 39th international conference on parallel processing workshops*. IEEE, 207–216.

[47] Jaylen Wang, Daniel S Berger, Fiodar Kazhamiaka, Celine Irvene, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warrier, Chetan Bansal, et al. 2024. Designing cloud servers for lower carbon. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 452–470.

[48] Jaylen Wang, Udit Gupta, and Akshitha Sriraman. 2023. Peeling Back the Carbon Curtain: Carbon Optimization Challenges in Cloud Computing. In *Proceedings of the 2nd Workshop on Sustainable Computer Systems*. 1–7.

[49] Hanfei Yu, Rohan Basu Roy, Christian Fontenot, Devesh Tiwari, Jian Li, Hong Zhang, Hao Wang, and Seung-Jong Park. 2024. RainbowCake: Mitigating Cold-starts in Serverless with Layer-wise Container Caching and Sharing. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 335–350.